

Módulo 3: Identidad y Control de Acceso

T08: IDOR y Control de Acceso

Carlos Rodríguez Contreras · GitHub: karrocon

 *Acceder a recursos de otros usuarios manipulando parámetros*

Objetivos

- Entender qué es IDOR (Insecure Direct Object Reference) y por qué es tan frecuente
- Explotar el endpoint de perfil de VulnApp para ver datos de otros usuarios
- Distinguir escalada horizontal (otro usuario) vs. vertical (otro rol)
- Implementar verificación de pertenencia de recursos en el servidor

¿Qué es IDOR?

Insecure Direct Object Reference — acceder a recursos de otros usuarios simplemente cambiando un identificador en la petición.

```
GET /users/1      → Mi perfil (OK)
GET /users/2      → Perfil de otro usuario (¡sin verificación!)
PUT /products/5   → Editar coche de otro usuario (¡sin verificación!)
```

💀 **OWASP A01:2021 — Broken Access Control:** presente en el 94% de aplicaciones analizadas. Es la vulnerabilidad #1 del ranking.

Escalada horizontal vs. vertical

Tipo	Descripción	Ejemplo
Horizontal	Acceder a recursos de otro usuario del mismo nivel	User A lee datos de User B
Vertical	Acceder a funciones de un rol superior	User accede a funciones de Admin

En VulnApp ambas son posibles:

- Horizontal: `GET /users/2` (bob ve datos de alice)
- Vertical: `DELETE /products/1` (user borra sin ser admin)

El código vulnerable en VulnApp

```
// src/routes/users.ts - línea 10
// ⚠️ NO verifica que el usuario autenticado sea el dueño del recurso
router.get('/:id', (req, res) => {
  const id = parseInt(req.params.id, 10);
  const user = db.prepare('SELECT id, username, email, role FROM users WHERE id = ?').get(id);
  res.json(user); // ← Devuelve datos de CUALQUIER usuario
});
```

```
// src/routes/products.ts - línea 62
// ⚠️ Cualquier usuario autenticado puede editar CUALQUIER producto
router.put('/:id', requireAuth, (req, res) => {
  // No verifica: existing.owner_id === req.user.userId
  db.prepare('UPDATE products SET ...').run(...);
});
```

Demo: IDOR en perfiles

```
# Login como bob (userId: 2)
TOKEN=$(curl -s -X POST http://localhost:3000/login \
  -H "Content-Type: application/json" \
  -d '{"username":"bob","password":"qwerty"}' | jq -r .token)

# Ver mi perfil (OK)
curl http://localhost:3000/users/2 -H "Authorization: Bearer $TOKEN"

# Ver perfil de alice (¡NO debería funcionar!)
curl http://localhost:3000/users/1 -H "Authorization: Bearer $TOKEN"
# → Devuelve: {"id":1,"username":"alice","email":"alice@example.com","role":"admin"}
```

Demo: IDOR en productos

```
# Bob edita un coche de Alice (owner_id=1)
curl -X PUT http://localhost:3000/products/1 \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name":"HACKEADO"}'
# → 200 OK - "Producto actualizado" 🍷
```

⚠ El servidor no verifica que bob sea el propietario del producto 1.

El fix: verificación de pertenencia

```
// src/routes/users.ts - fix
router.get('/:id', requireAuth, (req: AuthRequest, res) => {
  const id = parseInt(req.params.id, 10);

  // ✅ Solo puedes ver tu propio perfil (o ser admin)
  if (req.user!.userId !== id && req.user!.role !== 'admin') {
    return res.status(403).json({ error: 'Acceso denegado' });
  }
  // ...
});

// src/routes/products.ts - fix
router.put('/:id', requireAuth, (req: AuthRequest, res) => {
  const existing = db.prepare('SELECT * FROM products WHERE id = ?').get(id);

  // ✅ Solo el propietario o admin pueden editar
  if (existing.owner_id !== req.user!.userId && req.user!.role !== 'admin') {
    return res.status(403).json({ error: 'No eres el propietario' });
  }
  // ...
});
```


Principios de control de acceso

Principio	Aplicación
Denegar por defecto	Todo prohibido salvo lo explícitamente permitido
Verificar en servidor	Nunca confiar en checks del cliente
Mínimo privilegio	Cada rol solo accede a lo que necesita
IDs opacos	UUIDs en vez de enteros secuenciales (dificulta enumeración)
Log de accesos	Registrar intentos de acceso denegado (detección)

IDs secuenciales vs. UUIDs

Secuencial: /users/1, /users/2, /users/3 ← fácil de enumerar
UUID: /users/550e8400-e29b-41d4-a716-446655440000 ← no predecible

Aspecto	ID Secuencial	UUID v4
Predecibilidad	Alta (n+1)	Prácticamente imposible
Enumeración	Trivial (1,2,3...)	Requiere otra fuga
Rendimiento	Mejor índice	Ligeramente peor
Seguridad	No es defensa (sigue necesitando authz)	Defensa en profundidad

 **UUID no es un control de acceso** — es solo una capa extra. Siempre verifica permisos en servidor.

Casos reales de IDOR

Año	Plataforma	Vector	Impacto
2018	Facebook	<code>GET /albums?id=X</code> sin authz	Fotos privadas de cualquier usuario
2019	First American	URLs con ID secuencial	885M documentos financieros expuestos
2020	Parler	API con IDs secuenciales sin auth	Descarga masiva de posts (incluidos borrados)
2022	T-Mobile	Endpoint interno con ID de cliente	37M registros de clientes

💀 IDOR es la vulnerabilidad más frecuente en programas de bug bounty — y una de las más fáciles de encontrar.



Lab T08: IDOR en perfiles de usuario

Objetivo: acceder al perfil de otro usuario manipulando la URL

Setup: `cd VulnApp && npm start`

1. Inicia sesión como usuario 1 y navega a `GET /users/1`
2. Cambia el ID: `GET /users/2` — ¿puedes ver los datos del usuario 2?
3. Abre `src/routes/users.ts` y añade verificación: `req.user.id === userId`
4. Verifica que el acceso a otro perfil ahora devuelve 403